

Angular Avançado · Módulos 3, 6 e 7

Sessão 9

Stores, guards e interceptors

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficámos na Sessão 8

Providers e scopes

Providers

- `{ provide, useClass }`
- Provider functions: `provide…()`
- `useValue` · `InjectionToken`

Scopes

- Root → rota → componente
- O mais próximo **ganha** · `skipSelf`
- Auto cleanup nas rotas

No código

- `LanguageService`: contrato + duas implementações
- `provideLanguageService('browser')` no root
- `default` no About e na rota `booking`

Por resolver: ir a About e voltar ainda apaga a pesquisa. Hoje começamos por aí — com uma **store**.

Agenda de hoje

01	**Stores**	Estado que sobrevive à navegação · linkedSignal	≈ 35 min
02	**Guards funcionais**	canActivate · canMatch · canDeactivate	≈ 55 min
03	**Interceptors funcionais**	Juntar o token · a ordem da cadeia (se o tempo deixar)	≈ 30 min
04	**Resumo**	E o que vem na Sessão 10: Signal Forms	≈ 5 min



Bloco D · 5.2

Estado que sobrevive à navegação

Stores: serviços que guardam estado em signals

Store: um serviço que guarda estado

FlightSearch
criado e destruído a cada visita



FlightStore (root)
vive enquanto a app viver



FlightClient
acesso a dados

O que vai para a store

- Filtro (`from`, `to`) e cesto
- Os voos carregados (o resource)
- Métodos para **alterar** o estado

Regras

- Escrita **só dentro** da store
- Para fora: signals **read-only** + métodos
- O componente fica com a **apresentação**



Da teoria ao código

Agora, vamos escrever o código

- A FlightStore: estado privado, leitura read-only, métodos com nomes de negócio
- linkedSignal: uma cópia local e editável do filtro
- O FlightSearch a usar a store — e a pesquisa sobrevive à navegação

Ficheiros: flight-store.ts · flight-search.ts

FlightStore: estado encapsulado em signals

feature-booking/flight-search/flight-store.ts

```
@Service()
export class FlightStore {
  private readonly flightClient = inject(FlightClient);

  private readonly _from = signal('Lisbon');       // escrita: cá dentro
  readonly from = this._from.asReadonly();          // leitura: para fora
  private readonly _to = signal('Porto');
  readonly to = this._to.asReadonly();
  private readonly _basket = signal<Record<number, boolean>>({ 3: true, 5: true });
  readonly basket = this._basket.asReadonly();

  private readonly flightsResource = this.flightClient.findResource(this.from, this.to);
  readonly flights = this.flightsResource.value;   // + isLoading · error · status

  updateFilter(from: string, to: string): void { this._from.set(from); this._to.set(to); }
  updateBasket(id: number, selected: boolean): void {
    this._basket.update((b) => ({ ...b, [id]: selected }));
  }
  reload(): void { this.flightsResource.reload(); }
}
```

linkedSignal: uma cópia local, ligada à fonte

```
protected readonly filter = linkedSignal(() => ({
  from: this.store.from(),
  to: this.store.to(),
}));

protected readonly filterForm = form(this.filter);   // o form escreve na cópia
```

	Deriva de outros signals?	Pode escrever-se?	Quando a fonte muda…
signal	não	sim	—
computed	sim	não (read-only)	recalcula
linkedSignal	sim	sim — só a cópia local	a cópia é substituída

Escrever no `linkedSignal` **não** altera a store. Para isso, chama-se um método da store.

O FlightSearch a usar a store

flight-search.ts — o template não muda

```
export class FlightSearch {
  private readonly store = inject(FlightStore);

  protected readonly filter = linkedSignal(() => ({
    from: this.store.from(), to: this.store.to(),
  }));
  protected readonly filterForm = form(this.filter);

  protected readonly flights = this.store.flights;   // + error · isLoading · status
  protected readonly basket = this.store.basket;
  protected readonly maxDelay = signal(0);           // só UI: fica aqui

  protected search(): void {
    this.store.updateFilter(this.filter().from, this.filter().to);
    this.store.reload();
  }
  protected updateBasket(id: number, selected: boolean): void {
    this.store.updateBasket(id, selected);
  }
}
```

Experimentar

- Pesquisar Faro → Porto, marcar um voo
- Ir a About e voltar: **fica tudo**

O preço

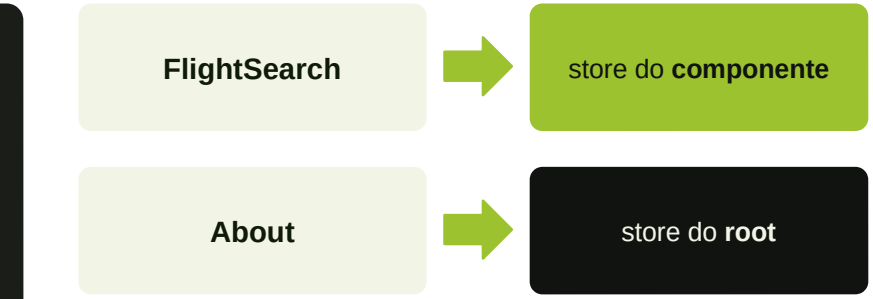
- Escrever no filtro já **não** dispara a pesquisa
- É preciso o botão **Search**

E se a store estiver no componente?

Demo opcional: a mesma ida ao About, com providers: [FlightStore]

flight-search.ts

```
@Component({
  selector: 'app-flight-search',
  templateUrl: './flight-search.html',
  providers: [FlightStore],   // só para a demo
})
export class FlightSearch {
  private readonly store = inject(FlightStore);
  …
```



Duas instâncias: a do componente **tapa** a do root.

Experimentar

- Faro → Porto → About → voltar
- Aparece **Lisbon** → **Porto**: valores iniciais

Conclusão

- O singleton vem do provider estar **só no root**
- Não do `@Service()` em si

Recuperar a reatividade

linkedSignal com set — e debounce no formulário

```
// 1 · a escrita no filtro vai para a store
protected readonly filter = linkedSignal(
  () => ({ from: this.store.from(), to: this.store.to() }),
  { set: (value) => this.store.updateFilter(value.from, value.to) },
);

protected search(): void {
  this.store.reload();          // o filtro já está na store
}
```

```
// 2 · sem isto: um pedido HTTP por tecla
protected readonly filterForm = form(this.filter, (path) => {
  debounce(path.from, 300);
  debounce(path.to, 300);
});
```

Experimentar

- Passo 1: escrever **Paris** → um pedido por tecla
- Passo 2: com `debounce` → **um só** pedido

Alternativas

- Signals graváveis na store
- `(input)` → `updateFilter`

O `debounce` atrasa a escrita no **modelo** do formulário: só então o `set` chama a store.

Onde registar a store: quanto tempo dura o estado

Scope	Como	O estado dura…	Bom para
Root	@Service()	enquanto a app viver (limpar à mão: reset())	retomar onde se estava — o cesto
Rota	providers na rota + auto cleanup	enquanto se estiver na feature	features com várias rotas (wizards)
Componente	providers no @Component	enquanto o componente existir	componentes complexos reutilizáveis

A pergunta certa não é "store ou não store" — é **quanto tempo deve viver este estado**.

A `FlightStore` fica no root: é isso que faz a pesquisa sobreviver à ida ao About.

A seguir: menos código repetido

Store feita à mão

- `_x = signal(...)` + `x = _x.asReadonly()` para cada propriedade
- Resource + `value` / `isLoading` / `error` à mão
- Um método por alteração

NgRx Signal Store

- `signalStore(withState, withComputed, withMethods, …)`
- Estado read-only para fora, por omissão
- Já está no `package.json` do Skyboard (`@ngrx/signals`)

Sessão 12 — NgRx Signal Store. Os conceitos de hoje (scopes, read-only, métodos) são os mesmos.

Resumo: serviços, DI e estado

Serviços e DI

- `@Service()` = singleton no root
- `inject(Token)` num **injection context**
- Token · provider · instância
- Serviços pedem serviços

Providers e scopes

- `{ provide, useClass }` · `provide…()`
- Root → rota → componente
- O mais próximo **ganha**
- Auto cleanup nas rotas

Stores

- Estado num serviço que não morre com o componente
- Privado + `asReadonly()` + métodos
- `linkedSignal` para editar uma cópia
- O scope decide o tempo de vida

Pergunta da Sessão 6 respondida: a pesquisa sobrevive porque o estado passou para uma store no root.

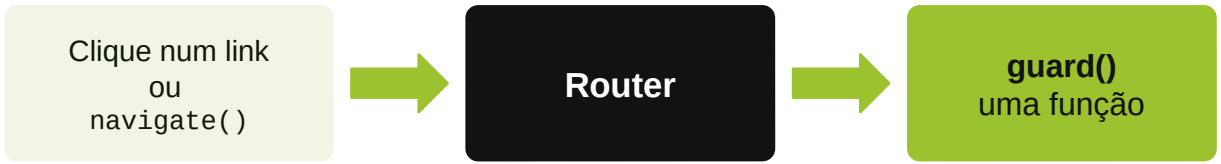


Bloco E · Guards

Guards funcionais

Funções que o router chama antes de mudar de rota

O que é um guard?



`true` → ativa a rota

`false` → cancela

`UrlTree` → redireciona

- Configura-se **na rota**: `canActivate: [authGuard]`
- É uma **função**, chamada num **injection context** → pode fazer `inject()`
- Pode decidir já, ou mais tarde (`Observable` / `Promise`): o router espera

Guards em classe (`implements CanActivate`) estão deprecated. Em código antigo: `mapToCanActivate([AuthGuard])`.

Quatro tipos de guard

Propriedade da rota	Tipo	Pergunta	Exemplo
`canActivate`	CanActivateFn	Pode **entrar**?	Só com sessão
`canActivateChild`	CanActivateChildFn	Pode entrar nas **filhas**?	Toda uma área de admin
`canMatch`	CanMatchFn	Esta rota **conta** para este URL?	Rotas por perfil · feature flags
`canDeactivate`	CanDeactivateFn<T>	Pode **sair**?	Alterações por guardar

Todos são **arrays**: a navegação só continua se **todos** os guards da rota disserem que sim.

O que um guard pode devolver

```ts
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthStore);          // injection context
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;                           // segue
  }
  return router.createUrlTree(['/home']);  // redireciona
};
```

Os argumentos (`route`, `state`) existem para quando a decisão depende do URL — por exemplo, um parâmetro.

Valores possíveis

- `true` — segue
- `false` — cancela (sem feedback)
- `UrlTree` — redireciona
- `RedirectCommand` — redireciona com opções
- `Observable` / `Promise` de um destes — o router **espera**

Um guard não é segurança

O guard (frontend)

- Corre no browser do utilizador
- Melhora a **experiência**: não mostra ecrãs que vão falhar
- Leva o utilizador ao sítio certo (o login)

A API (backend)

- É quem **protege** os dados
- Valida o token em **cada** pedido
- Responde `401` / `403` quando não deve

Tudo o que está no frontend pode ser lido e alterado nas DevTools. A regra de negócio vive no servidor.



Da teoria ao código

Agora, vamos escrever o código

- Uma `AuthStore` (a receita da `FlightStore`)
- `canActivate` com `false`, depois com `UrlTree`
- `canMatch` (com uma segunda rota) e `canDeactivate`

shared/util-auth/ · shared/util-common/exit.guard.ts · ticketing.routes.ts

Uma store para a sessão do utilizador

domains/shared/util-auth/auth-store.ts

```
@Service()
export class AuthStore {
  private readonly _userName = signal<string | null>(null);
  readonly userName = this._userName.asReadonly();

  readonly isLoggedIn = computed(() => this._userName() !== null);
  readonly token = computed(() => {      // fictício
    const userName = this._userName();
    return userName ? `demo-token-${userName.toLowerCase()}` : null;
  });

  login(userName: string): void { this._userName.set(userName); }
  logout(): void { this._userName.set(null); }
}
```

```
<!-- navbar.html -->
@if (auth.isLoggedIn()) {
  <span>{{ auth.userName() }}</span>
  <button (click)="auth.logout()">
    Logout</button>
} @else {
  <button
(click)="auth.login('Maria')">
    Login</button>
}
```

Na `Navbar`: `protected readonly auth = inject(AuthStore);` · CSS: `.navbar-actions button` a branco

canActivate: o guard mais simples

Passo 1 — devolver true ou false

```ts
// shared/util-auth/auth.guard.ts
export const authGuard: CanActivateFn = () =>
  inject(AuthStore).isLoggedIn();
```

```ts
// ticketing.routes.ts
children: [
  { path: 'flight-search', component: FlightSearch },
  { path: 'flight-edit/:id', component: FlightEdit, canActivate: [authGuard] },
  …
],
```

Sem sessão

- Clicar em **Edit** → não acontece nada

O problema

- O utilizador não sabe **porquê**

canActivate: redirecionar com um UrlTree

Passo 2 — levar o utilizador ao sítio certo

auth.guard.ts

```ts
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthStore);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;
  }
  return router.createUrlTree(['/home'], {
    queryParams: { needsLogin: true },
  });
};
```

```ts
// home.ts — chega pela query string
readonly needsLogin = input(false, {
  transform: booleanAttribute,
});
```

```html
<!-- home.html -->
@if (needsLogin()) {
  <p><strong>Please log in
    to continue.</strong></p>
}
```

O `input()` recebe o `?needsLogin=true` graças ao `withComponentInputBinding()` da Sessão 7.

canMatch: a rota só existe para alguns

```
// auth.guard.ts
export const authMatch: CanMatchFn = () =>
  inject(AuthStore).isLoggedIn();

// ticketing.routes.ts — o mesmo path duas vezes; a ordem conta
{ path: 'boarding', component: BoardingList, canMatch: [authMatch] },
{ path: 'boarding', component: BoardingLocked },   // a "seguinte"
```

	canActivate a false	canMatch a false
O que faz o router	cancela a navegação	ignora a rota e **continua a procurar**
Na Skyboard	fica onde estava	mostra a `BoardingLocked` — **mesmo URL**
Bom para	"precisas de login"	a mesma rota com versões diferentes (perfil, flags)

O `canMatch` não impede o preloading: decide o **match**, não o download.

canDeactivate: antes de perder alterações

```ts
// shared/util-common/exit.guard.ts
export interface DirtyAware {             // o contrato
  isDirty(): boolean;
}

export const exitGuard: CanDeactivateFn<DirtyAware> = (component) => {
  if (!component.isDirty()) {
    return true;
  }
  return confirm('Do you really want to leave without saving?');
};
```

Notar

- Recebe a **instância** do componente
- Uma `interface` chega: aqui é só um **tipo**, não um token de DI
- `confirm()`: para a demo

Na rota: `canDeactivate: [exitGuard]` — ao lado do `canActivate: [authGuard]`.

O FlightEdit diz se tem alterações

feature-booking/flight-edit/flight-edit.ts

```ts
export class FlightEdit implements DirtyAware {
  readonly id = input.required({ transform: numberAttribute });
  readonly showDetails = input(false, { transform: booleanAttribute });

  // por agora simulado; com as Signal Forms: this.flightForm().dirty()
  protected readonly dirty = signal(false);

  isDirty(): boolean {
    return this.dirty();
  }
}
```

Experimentar

- Login → **Edit**
- **Simulate a change**
- **Back to search** → pergunta
- Sem alterações → sai logo

No template: `<button (click)="dirty.set(true)">Simulate a change</button>`. Sai o código de demo da Sessão 7.

Na vida real: um diálogo em vez do confirm()

Referência — um guard assíncrono: o router espera

```ts
export const exitGuard: CanDeactivateFn<DirtyAware> = (component) => {
  if (!component.isDirty()) {
    return true;
  }
  const dialogRef = inject(Dialog).open<boolean>(ConfirmDialog, {
    data: 'Do you really want to leave without saving?',
  });
  return dialogRef.closed.pipe(map((result) => result === true));
};
```

```ts
// confirm-dialog.ts
message =
inject<string>(DIALOG_DATA);
dialogRef =
inject<DialogRef<boolean>>(
  DialogRef);

close(decision: boolean) {
  this.dialogRef.close(decision);
}
```

`DIALOG_DATA`: quem cria o componente dá-lhe um injector com os dados (ver o anexo: um Injector à medida).

Código antigo: guards em classe

Referência — ligar sem reescrever, com mapTo…

antes · legacy-auth.guard.ts

```ts
@Injectable({ providedIn: 'root' })
export class LegacyAuthGuard implements CanActivate {
  private readonly auth = inject(AuthStore);
  private readonly router = inject(Router);

  canActivate(route, state): boolean | UrlTree {
    return this.auth.isLoggedIn()
      || this.router.createUrlTree(['/home']);
  }
}
```

agora · ticketing.routes.ts

```ts
// rota: liga a classe sem a reescrever
canActivate: mapToCanActivate([LegacyAuthGuard])

// o que faz por dentro
canActivate: [(route, state) =>
  inject(LegacyAuthGuard).canActivate(route, state)]

// misturar com guards funcionais
[...mapToCanActivate([LegacyAuthGuard]), authGuard]
```

Família: `mapToCanActivate` · `mapToCanActivateChild` · `mapToCanDeactivate` · `mapToCanMatch` · `mapToResolve`

Código novo: **funções** (G2–G4). Código existente: `mapTo…` e migrar aos poucos.

Variante: voltar onde estava

Referência — Login na mensagem da Home + returnUrl

```ts
// auth.guard.ts — state.url = o URL bloqueado
export const authGuard: CanActivateFn = (route, state) => {
  …
  return router.createUrlTree(['/home'], {
    queryParams: { needsLogin: true, returnUrl: state.url },
  });
};
```

```ts
// home.ts
readonly returnUrl = input<string>();

protected login(): void {
  this.auth.login('Maria');
  this.router.navigateByUrl(
    this.returnUrl() ?? '/home');
}
```

```html
<!-- home.html -->
@if (needsLogin()) {
  <p><strong>Please log in to continue.</strong>
     <button (click)="login()">Login</button></p>
}
```

Porquê

- Só o botão não chega: o `needsLogin` fica no URL
- Validar que o `returnUrl` é **interno**

Guards: resumo

canActivate

- Pode **entrar**?
- `false` cancela · `UrlTree` redireciona

canMatch

- A rota **conta**?
- `false` → o router continua à procura

canDeactivate

- Pode **sair**?
- Recebe a instância do componente

Funções num **injection context** → `inject()`. Ajudam a UX; a segurança está na API.

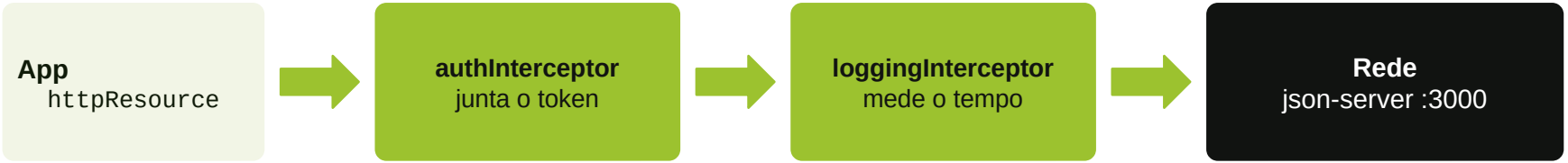


Bloco F · Interceptors

Interceptors funcionais

Funções entre a app e a rede

Uma cadeia entre a app e a rede



`pedido →` `← resposta (ordem inversa)`

- Vê **todos** os pedidos do `HttpClient` — incluindo os do `httpResource`
- Cada um recebe o pedido e o `next`: mexe no pedido e passa-o ao seguinte
- Usos típicos: **token**, erros `401`/`403` num só sítio, logging, loading global, retry, cache

Anatomia de um interceptor

```
export const myInterceptor: HttpInterceptorFn = (req, next) => {
  const store = inject(AuthStore);              // injection context

  // ANTES do next: o pedido (imutável → clone)
  const newReq = req.clone({ setHeaders: { 'X-App': 'skyboard' } });

  // o next: passa ao seguinte da cadeia
  return next(newReq).pipe(
    // DEPOIS do next: a resposta (tap, catchError, finalize…)
    catchError((error) => { /* … */ return throwError(() => error); }),
  );
};
```

O `HttpClient` é baseado em **Observables** — os interceptors também, mesmo quando a app usa `httpResource`.

Registar: provideHttpClient(withInterceptors(…))

```
// app.config.ts
providers: [
  provideRouter(routes, …),
  provideHttpClient(
    withInterceptors([authInterceptor, loggingInterceptor]),
  ),
  provideLanguageService('browser'),
],
```

É uma **provider function** — o mesmo padrão do `provideLanguageService` da Sessão 8.

Notar

- Sem `provideHttpClient` o `HttpClient` já existe (Angular 22) — mas para **configurar** é preciso
- A ordem do array **é** a ordem da cadeia
- Código antigo (classes): `withInterceptorsFromDi()`



Da teoria ao código

Agora, vamos escrever o código

- Um interceptor que junta o token da `AuthStore`
- Um segundo, de logging — e trocar a ordem para ver a cadeia
- Ver os headers na Network e os logs na consola

shared/util-auth/auth.interceptor.ts · shared/util-common/logging.interceptor.ts · app.config.ts

Juntar o token aos pedidos da nossa API

domains/shared/util-auth/auth.interceptor.ts

```ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthStore).token();
  const apiUrl = inject(ConfigService).apiUrl;

  // sem sessão, ou pedido para outro servidor: segue tal como está
  if (!token || !req.url.startsWith(apiUrl)) {
    return next(req);
  }

  // os pedidos são imutáveis: clona-se com o header novo
  const authReq = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  return next(authReq);
};
```

O interceptor vê **todos** os pedidos da app. O `startsWith(apiUrl)` garante que o token não vai para terceiros.

Registar e ver na Network

```
// app.config.ts
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from './domains/shared/util-auth/auth.interceptor';

provideHttpClient(withInterceptors([authInterceptor])),
```

Sem sessão

- Network → `/flights` → Request Headers
- Sem `Authorization`

Com sessão

- **Login → Search**
- `Authorization: Bearer demo-token-maria`

Um segundo interceptor: a ordem conta

shared/util-common/logging.interceptor.ts

```ts
export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const started = performance.now();
  const auth = req.headers.has('Authorization') ? 'com token' : 'sem token';
  console.log(`→ ${req.method} ${req.urlWithParams} (${auth})`);

  return next(req).pipe(                // depois do next: a resposta
    finalize(() => {
      const ms = Math.round(performance.now() - started);
      console.log(`← ${req.method} ${req.urlWithParams} · ${ms} ms`);
    }),
  );
};
```

[auth, logging]

- Consola: `(com token)`

[logging, auth]

- Consola: `(sem token)`

A ordem do array é a ordem no **pedido**; na resposta é a inversa.

Para saber: erros 401/403 num só sítio

```ts
export const authErrorInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthStore);
  const router = inject(Router);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === HttpStatusCode.Unauthorized
          || error.status === HttpStatusCode.Forbidden) {
        auth.logout();
        router.navigate(['/home'], { queryParams: { needsLogin: true } });
      }
      return throwError(() => error);   // quem pediu também sabe
    }),
  );
};
```

Outros usos

- Loading global (contar pedidos ativos)
- `retry`
- Cache

Resumo da sessão

AuthStore

- A receita da `FlightStore`
- Serve a Navbar, o guard e o interceptor

Guards

- `canActivate` · `canMatch` · `canDeactivate`
- `true` · `false` · `UrlTree` · async
- UX, não segurança

Interceptors

- `(req, next)` · `clone()`
- `provideHttpClient(withInterceptors([…]))`
- A ordem do array é a cadeia

Funções que o Angular chama no momento certo, num **injection context** — por isso fazem `inject()`.



Na Sessão 10

Signal Forms

- O `FlightEdit` com um formulário a sério
- Validação no schema · erros · submit
- O `isDirty()` de hoje passa a ser o `dirty()` do formulário



Anexo · opcional

Um Injector à medida

Criar um contexto de DI só para um componente — e passar-lhe dados sem input()

Duas coisas que se fazem com um Injector

Entrar num contexto que existe

- `runInInjectionContext(injector, () => inject(…))`
- Já o vimos: o Try inject da Sessão 7
- Ou passar o injector: `effect(fn, { injector })`, `toSignal(obs$, { injector })`

Criar um injector novo

- `Injector.create({ providers, parent })`
- Um **filho**: o que não tem, pede ao pai
- `createEnvironmentInjector(…)` para `provide…()` — e `destroy()` no fim



É o padrão dos dialogs: o `DIALOG_DATA` do Angular CDK chega assim ao componente.



Da teoria ao código

Agora, vamos escrever o código

- Um token FLIGHT e um FlightDetails que o pede com inject()
- Um injector filho com o voo escolhido
- Criar o FlightDetails em runtime com esse injector

Ficheiros: flight-details.ts · flight-search.ts · flight-search.html

Um componente que recebe o voo pela DI

```ts
// ui/flight-details/flight-details.ts
export const FLIGHT = new InjectionToken<Flight>('FLIGHT');

@Component({
  selector: 'app-flight-details',
  template: `
    <h3>#{{ flight.id }} · {{ flight.from }} → {{ flight.to }}</h3>
    <p>{{ flight.date }} · delay: {{ flight.delay }} min</p>`,
})
export class FlightDetails {
  protected readonly flight = inject(FLIGHT);   // em vez de input()
}
```

```html
<!-- flight-search.html -->
<ng-container #details />
<button type="button" class="actions"
        (click)="showDetails(flight)">Details</button>
```

Sozinho, não funciona

- Ninguém fornece `FLIGHT` → `NG0201`
- Quem o **cria** tem de lhe dar um injector com o voo
- O `#details` marca onde vai aparecer

Um injector filho, feito à mão

```ts
// flight-search.ts
private readonly injector = inject(Injector);
private readonly details =
  viewChild.required('details', { read: ViewContainerRef });

protected showDetails(flight: Flight): void {
  const child = Injector.create({
    providers: [{ provide: FLIGHT, useValue: flight }],
    parent: this.injector,          // o injector deste componente
  });

  this.details().clear();
  this.details().createComponent(FlightDetails, { injector: child });
}
```

Cuidados

- O **pai** conta: com o do componente, vê a store e a rota
- `Injector.create`: só providers simples
- `createEnvironmentInjector` → `destroy()` à mão

Dados sem `input()`: quem cria o componente decide o que ele recebe — é DI, outra vez.